

PRÁCTICA OPCIONAL

Sistemas distribuidos

SIMULADOR DE BLOCKCHAIN



12/05/2026

Autores

Alfonso Crego Calvo

Diego Martínez Santamaría

INTRODUCCIÓN

La práctica opcional escogida consiste en un pequeño simulador de Blockchain, que permite realizar las funciones principales y esenciales para el correcto funcionamiento del mismo. El objetivo principal, es poder comprender correctamente cómo es que se realizan transacciones, se minan bloques y resuelven posibles conflictos, basándose en la manera de proceder de Bitcoin.

Para poder visualizar correctamente todo este procedimiento, se ha optado por implementar utilidades de Swagger. Este es un conjunto de herramientas y especificaciones que en este caso, se utilizará para poder exponer endpoints REST en el navegador web. Este está a su vez conectado al código fuente desarrollado, por lo que los atributos y métodos que se ejecuten desde el navegador serán directamente los que se pueden apreciar en el programa y clases del mismo.

PASOS A SEGUIR

Para poder comenzar con la práctica, es necesario contar con tres equipos que se encuentren en la misma red. Se podrá realizar también con tres terminales ejecutadas en el mismo ordenador, aunque para poder mostrar que funciona en sistemas distribuidos, se ejemplificará de la primera manera propuesta.

Procederemos con el siguiente comando ejecutado en la terminal de cada equipo. Anotar que se tendrá que encontrar en el directorio donde se encuentra el programa para poder ejecutarlo exitosamente:

```
dotnet run --urls "http://<IP-equipo>:8080"
```

Una vez ejecutado, si en el navegador se introduce la url: <http://localhost:8080/swagger>, se podrá comprobar la interfaz que nos presenta Swagger con todos los endpoints REST expuestos, como se comentaba anteriormente.

PASO 1 : REGISTRAR NODOS

El primer paso que se tendrá que realizar, es hacer click en el método registrar para introducir el resto de nodos vecinos. Hacemos por tanto click en "Try Out", y modificamos el código como en la imagen presentada a continuación:

The screenshot shows a REST client interface with the following sections:

- POST /blockchain/nodos/registrar**: The endpoint and method.
- Parameters**: A section with "No parameters" and buttons for "Cancel" and "Reset".
- Request body**: A text area containing a JSON object:

```
{  "nodos": [    "http://192.168.1.34:8080"  ]}
```

 A dropdown menu shows "application/json".
- Execute**: A blue button to execute the request.
- Responses**: A section containing:
 - Curl**: A terminal window showing the equivalent curl command:

```
curl -X 'POST' \  'http://192.168.1.36:8080/blockchain/nodos/registrar' \  -H 'accept: */*' \  -H 'Content-Type: application/json' \  -d '{  "nodos": [    "http://192.168.1.34:8080"  ]}'
```
 - Request URL**: A text box showing `http://192.168.1.36:8080/blockchain/nodos/registrar`.
 - Server response**: A section with a "Code" of 200 and a "Details" link.
 - Response body**: A text area showing the JSON response:

```
{  "mensaje": "NODOS REGISTRADOS:",  "nodos": [    "http://192.168.1.34:8080"  ]}
```

 A "Download" button is present.
 - Response headers**: A text area showing headers:

```
content-type: application/json; charset=utf-8  date: Thu, 14 May 2026 14:27:59 GMT  server: Kestrel  transfer-encoding: chunked
```

El texto se encuentra en formato JSON debido a que al pulsar “Execute”, se crean directamente los objetos correspondientes (hay en el código un constructor vacío por ese mismo motivo). Este mismo procedimiento se tendrá que realizar con el resto de equipos para así rellenar la lista de nodos conocidos y poder ejecutar la resolución de conflictos en pasos posteriores.

PASO 2 : NUEVA TRANSACCIÓN

A continuación, se pasará a ejecutar el método que crear un nuevo objeto de transacción, con los siguientes campos rellenos como se desee:

POST

/blockchain/transaccion/nueva

Parameters

Cancel

Reset

No parameters

Request body

application/json

```
{
  "remite": "Alfonso",
  "receptor": "Diego",
  "cantidad": 350,
  "momento": "2026-05-14T14:30:13.132Z"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://192.168.1.36:8080/blockchain/transaccion/nueva' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "remite": "Alfonso",
    "receptor": "Diego",
    "cantidad": 350,
    "momento": "2026-05-14T14:30:13.132Z"
  }'
```

Request URL

http://192.168.1.36:8080/blockchain/transaccion/nueva

Server response

Code

Details

200

Response body

```
{
  "mensaje": "LA TRANSACCION SE ENCUENTRA EN EL BLOQUE 1."
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Thu, 14 May 2026 14:30:33 GMT
server: Kestrel
transfer-encoding: chunked
```

PASO 3 : MINAR BLOQUE

Ahora que ya se ha creado una transacci3n, se puede por tanto minar el bloque en el que se encontrar3 dicha transacci3n recientemente realizada. Al igual que ocurre en una Blockchain, si ejecutamos el m3todo "cadena", contemplaremos que s3lo est3 actualmente el bloque g3nesis, el primero de toda la lista. Volviendo al minado, se pondr3 el nombre del nodo que realiza la operaci3n y a continuaci3n en la respuesta devuelta, se podr3 comprobar que el bloque con 3ndice correcto se ha generado adecuadamente. Este tiene una referencia al hash del bloque anterior, enlazando estos objetos y formando as3 una cadena de bloques. Se ejecuta el comando con el siguiente JSON:

POST /blockchain/minar

Parameters

No parameters

Request body

application/json

```
{
  "minero": "Minero1"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://192.168.1.36:8080/blockchain/minar' \
  -H 'accept: */*' \
  -H 'content-type: application/json' \
  -d '{
    "minero": "Minero1"
  }'
```

Request URL

http://192.168.1.36:8080/blockchain/minar

Server response

Code

Details

200

Response body

```
{
  "mensaje": "BLOQUE MINADO",
  "indice": 1,
  "hash": "900F3138E51722AD03644AA17780E2F81F06D468FD3C8EC3B19C1EABE8986CD",
  "transacciones": [
    {
      "remite": "Alfonso",
      "receptor": "Diego",
      "cantidad": 350,
      "momento": "2026-05-14T14:38:13.132Z"
    },
    {
      "remite": "SISTEMA",
      "receptor": "Minero1",
      "cantidad": 10,
      "momento": "2026-05-14T16:32:40.3015866+02:00"
    }
  ]
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Thu, 14 May 2026 14:32:40 GMT
server: Kestrel
transfer-encoding: chunked
```

PASO 4 : RESOLUCIÓN DE CONFLICTOS

Para que la cadena sea coherente en todos los nodos, se sigue un sistema idéntico al de Bitcoin. El nodo que lo ejecute, comparará la cantidad de bloques que posee. En caso de que este encuentre una cadena más larga, significa que no estará actualizado a la situación actual de la aplicación, por lo la adoptará para no ir retrasado con respecto al estado del resto de nodos. Ejecutaremos por tanto el método de la siguiente forma:

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** /blockchain/nodos/resolverConflictos
- Parameters:** No parameters
- Execute:** Button to execute the request
- Clear:** Button to clear the request
- Responses:** Section containing:
 - Curl:** `curl -X 'GET' \ 'http://192.168.1.34:8080/blockchain/nodos/resolverConflictos' \ -H 'accept: */*'`
 - Request URL:** `http://192.168.1.34:8080/blockchain/nodos/resolverConflictos`
 - Server response:**
 - Code:** 200
 - Response body:**

```
{
  "mensaje": "LA CADENA HA SIDO REEMPLAZADA POR UNA MÁS LARGA.",
  "cadena": [
    {
      "indice": 0,
      "momento": "2026-05-14T16:27:59.1688055+02:00",
      "transacciones": [],
      "anteriorHash": "0",
      "hash": "90CA90BDE629CC1F71F72B0C6905171FEDC332198B73D10644AE69AE0D2A4F20",
      "nonce": 0
    },
    {
      "indice": 1,
      "momento": "2026-05-14T16:32:40.3016362+02:00",
      "transacciones": [
        {
          "remite": "Alfonso",
          "receptor": "Diego",
          "cantidad": 350,
          "momento": "2026-05-14T14:30:13.132Z"
        },
        {
          "remite": "SISTEMA",
          "receptor": "Mineral",
          "cantidad": 10,
          "momento": "2026-05-14T16:32:40.3015866+02:00"
        }
      ],
      "anteriorHash": "90CA90BDE629CC1F71F72B0C6905171FEDC332198B73D10644AE69AE0D2A4F20",
      "hash": "20965138F133AD03644AA173805308160D46ED335F3810C14E5608FCD"
    }
  ]
}
```
 - Response headers:**

```
content-type: application/json; charset=utf-8
date: Thu, 14 May 2026 14:32:19 GMT
server: Kestrel
transfer-encoding: chunked
```

En caso de que por ejemplo, haya empate entre dos nodos con el mismo número de bloques, se adoptará la cadena de aquel nodo que se encuentre antes. Dependerá por tanto del orden en el que se haya registrado los nodos en el primer paso. Bitcoin por ejemplo, espera algo de tiempo para que alguno de esos nodos mine otro bloque y por tanto se resuelva este empate.

Si en este punto se comprueba la cadena de todos los nodos después de resolver todos los conflictos, se podrá apreciar que es la misma, consiguiendo que estén adecuadamente sincronizados.

Asimismo, si se desea comprobar el saldo introduciendo el nombre de las personas involucradas en las transacciones, se podrá comprobar que es coherente con respecto a los movimientos realizados. Cabe recalcar que se podrían realizar más transacciones, minar más bloques o repetir los pasos especificados para obtener una estructura más compleja.

Sin embargo, estos son los esenciales para poder comprender cómo este procedimiento se realizaría por N número de nodos, formando así una Blockchain más cercana a lo que ocurre hoy en día en las organizaciones que las usan.